

WEEK-1 PRACTICAL

1. Id:

```
the_boss@DESKTOP-I8C8AJG:~$ id
uid=1000(the_boss) gid=1000(the_boss) groups=1000(the_boss),27(sudo),30(dip),46(plugdev),100(users)
```

Report: I used the `id` command to display information about the current user. It showed my user ID, group ID and the groups associated with my user account.

2. Env:

```
the_boss@DESKTOP-I8C8AJG:~$ env
SHELL=/bin/bash
WSL2_GUI_APPS_ENABLED=1
WSL_DISTRO_NAME=Ubuntu
NAME=DESKTOP-I8C8AJG
PWD=/home/the_boss
LOGNAME=the_boss
HOME=/home/the_boss
LANG=C.UTF-8
WSL_INTEROP=/run/WSL/1934_interop
LS_COLORS=rs=0:di=01;34:ln=01;36:mh=00:pi=40;33:so=01;35
44:ex=01;32:*.tar=01;31:*.tgz=01;31:*.arc=01;31:*.arj=01
:*.t7z=01;31:*.zip=01;31:*.z=01;31:*.dz=01;31:*.gz=01;31
=01;31:*.tbz2=01;31:*.tz=01;31:*.deb=01;31:*.rpm=01;31:*
o=01;31:*.7z=01;31:*.rz=01;31:*.cab=01;31:*.wim=01;31:*
.gif=01;35:*.bmp=01;35:*.pbm=01;35:*.pgm=01;35:*.ppm=01;3
:*.mng=01;35:*.pcx=01;35:*.mov=01;35:*.mpg=01;35:*.mpeg=0
1;35:*.vob=01;35:*.qt=01;35:*.nuv=01;35:*.wmv=01;35:*.as
5:*.xcf=01;35:*.xwd=01;35:*.yuv=01;35:*.cgm=01;35:*.emf=0
36:*.mka=00;36:*.mp3=00;36:*.mpc=00;36:*.ogg=00;36:*.ra=0
old=00;90:*.orig=00;90:*.part=00;90:*.rej=00;90:*.swp=00
*.rpmnew=00;90:*.rpmorig=00;90:*.rpmsave=00;90:
WAYLAND_DISPLAY=wayland-0
LESSCLOSE=/usr/bin/lesspipe %s %s
TERM=xterm-256color
LESSOPEN=| /usr/bin/lesspipe %s
USER=the_boss
DISPLAY=:0
SHLVL=1
XDG_RUNTIME_DIR=/run/user/1000
WSLENV=
XDG_DATA_DIRS=/usr/local/share:/usr/share:/var/lib/snapd
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/s
porationII.WindowsSubsystemForLinux_2.7.11.0_x64__8wekyb3
es/Oracle/Java/java8path:/mnt/c/Program Files (x86)/Comm
nt/c/WINDOWS/System32/WindowsPowerShell/v1.0/:/mnt/c/WIND
ram Files/Git/cmd:/mnt/c/Program Files/nodejs/:/mnt/c/Use
Code/bin:/mnt/c/Users/ADMIN/AppData/Local/Python/bin:/m
DBUS_SESSION_BUS_ADDRESS=unix:path=/run/user/1000/bus
HOSTTYPE=x86_64
```

Report: I used the `env` command to display the environment variables available in my current shell. It helped me understand the different settings and variables used by the system.

3.export:

```

the_boss@DESKTOP-I8C8AJG:~$ export
declare -x DBUS_SESSION_BUS_ADDRESS="unix:path=/run/user/10
declare -x DISPLAY=":0"
declare -x HOME="/home/the_boss"
declare -x HOSTTYPE="x86_64"
declare -x LANG="C.UTF-8"
declare -x LESSCLOSE="/usr/bin/lesspipe %s %s"
declare -x LESSOPEN="| /usr/bin/lesspipe %s"
declare -x LOGNAME="the_boss"
declare -x LS_COLORS="rs=0:di=01;34:ln=01;36:mh=00:pi=40;33
34;42:st=37;44:ex=01;32:*.tar=01;31:*.tgz=01;31:*.arc=01;31
*.tzo=01;31:*.t7z=01;31:*.zip=01;31:*.z=01;31:*.dz=01;31:
=01;31:*.tbz=01;31:*.tbz2=01;31:*.tz=01;31:*.deb=01;31:*.rp
=01;31:*.cpio=01;31:*.7z=01;31:*.rz=01;31:*.cab=01;31:*.wim
jpeg=01;35:*.gif=01;35:*.bmp=01;35:*.pbm=01;35:*.pgm=01;35:
*.svgz=01;35:*.mng=01;35:*.pcx=01;35:*.mov=01;35:*.mpg=01;3
35:*.mp4v=01;35:*.vob=01;35:*.qt=01;35:*.nuv=01;35:*.wmv=0
35:*.dl=01;35:*.xcf=01;35:*.xwd=01;35:*.yuv=01;35:*.cgm=01;
5:*.midi=00;36:*.mka=00;36:*.mp3=00;36:*.mpc=00;36:*.ogg=00
ak=00;90:*.old=00;90:*.orig=00;90:*.part=00;90:*.rej=00;90
f-old=00;90:*.rpmnew=00;90:*.rpmorig=00;90:*.rpmsave=00;90:
declare -x NAME="DESKTOP-I8C8AJG"
declare -x OLDPWD
declare -x PATH="/usr/local/sbin:/usr/local/bin:/usr/sbin:/
MicrosoftCorporationII.WindowsSubsystemForLinux_2.7.11.0_x6
)/Common Files/Oracle/Java/java8path:/mnt/c/Program Files (C
em32/Wbem:/mnt/c/WINDOWS/System32/WindowsPowerShell/v1.0/:/
/mnt/c/Program Files/Git/cmd:/mnt/c/Program Files/nodejs:/
Microsoft VS Code/bin:/mnt/c/Users/ADMIN/AppData/Local/Pyth
declare -x PULSE_SERVER="unix:/mnt/wslg/PulseServer"
declare -x PWD="/home/the_boss"
declare -x SHELL="/bin/bash"
declare -x SHLVL="1"
declare -x TERM="xterm-256color"

```

Report: I used the `export` command to create or set an environment variable. The variable can then be accessed by commands and programs running from the current shell.

3. Alias:

```
the_boss@DESKTOP-I8C8AJG:~$ alias
alias alert='notify-send --urgency=low -i "${[ $? = 0 ]}$(echo error)" "$(history|tail -n1|sed -e '\''s/^\\s*[0-9]*\\s*//'\`')"'
alias egrep='egrep --color=auto'
alias fgrep='fgrep --color=auto'
alias grep='grep --color=auto'
alias l='ls -CF'
alias la='ls -A'
alias ll='ls -alF'
alias ls='ls --color=auto'
```

Report: I used the `alias` command to create a short name for a command. It helps me execute frequently used commands more easily without typing the complete command every time.

Task 1: Develop a C program that

demonstrates how a Linux operating system executes a command entered by a user that 1. Accept a Linux command as input. 2. Create a child process using `fork()`. 3. Execute the command in the child process using an appropriate `exec()` system call. 4. Allow the parent process to wait for the child using `wait ()`. 5. Display the Process ID

CODE:

pavani@DESKTOP-38OLEIN: ~

GNU nano 8.7.1

command_executor.c

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <string.h>

int main() {
    char command[100];
    pid_t pid;

    printf("Enter a Linux command: ");
    scanf("%s", command);

    pid = fork();

    if (pid < 0) {
        printf("Fork failed!\n");
        return 1;
    }

    else if (pid == 0) {
        printf("\nChild Process\n");
        printf("Child PID : %d\n", getpid());
        printf("Parent PID: %d\n", getppid());

        printf("\nExecuting command: %s\n\n", command);

        execlp(command, command, NULL);

        printf("Command execution failed!\n");
    }

    else {
        printf("\nParent Process\n");
        printf("Parent PID : %d\n", getpid());
        printf("Child PID : %d\n", pid);

        wait(NULL);

        printf("\nChild process completed.\n");
    }

    return 0;
}
```

OUTPUT:

```
pavani@DESKTOP-380LEIN: ~  
pavani@DESKTOP-380LEIN:~$ nano command_executor.c  
pavani@DESKTOP-380LEIN:~$ gcc command_executor.c -o command_executor  
pavani@DESKTOP-380LEIN:~$ ./command_executor  
Enter a Linux command: ls  
  
Parent Process  
Parent PID : 546  
Child PID : 547  
  
Child Process  
Child PID : 547  
Parent PID: 546  
  
Executing command: ls  
  
OSSP  command_executor  file2.txt  hello.sh      os-lab  shortcut  txt  
a.txt  command_executor.c  hello     helloCopy.java  ossp    snap  
b.txt  file1.txt           hello.c   main.java      script.sh  test.txt  
  
Child process completed.
```

Child process completed.

REPORT: I executed the C program successfully. The program accepted a Linux command from the user, created a child process using `fork()` for The `uname -a` command displayed complete information about my Linux system, including the kernel version, system architecture, hostname, and operating system details.`k()`, and executed the command using the `exec()` system call. The parent process waited until the child process finished execution. Finally, the program displayed the process IDs of both the parent and child processes, and the message "Child process completed." was displayed.

Task-2: Using Linux terminal commands (`uname`, `lscpu`, `ps`, `top`), investigate the relationship between hardware resources and operating system services. Prepare a report explaining how the OS abstracts CPU, memory, storage, and I/O devices.

Uname-a :

```
pavani@DESKTOP-380LEIN:~$ uname -a  
Linux DESKTOP-380LEIN 6.18.33.2-microsoft-standard-WSL2 #1 SMP PREEMPT_DYNAMIC Thu Jun 18 21:54:43 UTC 2  
026 x86_64 GNU/Linux
```

REPORT: The `uname -a` command displayed complete information about my Linux system, including the kernel version, system architecture, hostname, and operating system details.

Lscpu:

```
pavani@DESKTOP-380LEIN: ~
026 x86_64 GNU/Linux
pavani@DESKTOP-380LEIN:~$ lscpu
Architecture:                x86_64
CPU op-mode(s):              32-bit, 64-bit
Address sizes:                39 bits physical, 48 bits virtual
Byte Order:                   Little Endian
CPU(s):                       8
On-line CPU(s) list:         0-7
Vendor ID:                    GenuineIntel
Model name:                   11th Gen Intel(R) Core(TM) i5-1135G7 @ 2.40GHz
CPU family:                   6
Model:                        140
Thread(s) per core:           2
Core(s) per socket:           4
Socket(s):                    1
Stepping:                     1
BogoMIPS:                     4838.39
Flags:                         fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge mca cmov pat pse36 clf
                               lush mmx fxsr sse sse2 ss ht syscall nx pdpe1gb rdtscp lm constant_tsc arch
                               _perfmmon rep_good nopl xtopology tsc_reliable nonstop_tsc cpuid tsc_known_f
                               req pni pclmulqdq vmx ssse3 fma cx16 pdcmm pcid sse4_1 sse4_2 x2apic movbe p
                               opcnt tsc_deadline_timer aes xsave avx f16c rdrand hypervisor lahf_lm abm 3
                               dnowprefetch ssbd ibrs ibpb stibp ibrs_enhanced tpr_shadow ept vpid ept_ad
                               fsgsbase tsc_adjust bmi1 avx2 smep bmi2 erms invpcid avx512f avx512dq rdsee
                               d adx smap avx512ifma clflushopt clwb avx512cd sha_ni avx512bw avx512vl xsa
                               veopt xsavec xgetbv1 xsaves vnmix avx512vbmi umip avx512_vbmi2 gfni vaes vpc
                               lmulqdq avx512_vnni avx512_bitalg avx512_vpopcntdq rdpid movdiri movdir64b
                               fsrm avx512_vp2intersect md_clear ibt flush_l1d arch_capabilities

Virtualization features:
Virtualization:               VT-x
Hypervisor vendor:            Microsoft
Virtualization type:          full
Caches (sum of all):
L1d:                          192 KiB (4 instances)
L1i:                          128 KiB (4 instances)
L2:                           5 MiB (4 instances)
L3:                           8 MiB (1 instance)
NUMA:
NUMA node(s):                 1
NUMA node0 CPU(s):            0-7
Vulnerabilities:
Gather data sampling:          Unknown: Dependent on hypervisor status
Ghostwrite:                    Not affected
Indirect target selection:      Mitigation; Aligned branch/return thunks
Itlb multihit:                 Not affected
L1tf:                          Not affected
Mds:                           Not affected
Meltdown:                      Not affected
Mmio stale data:               Not affected
Old microcode:                 Not affected
Reg file data sampling:         Not affected
Spectre_v1:                    Mitigation; Enhanced IBPB
```

REPORT: The lscpu command displayed the CPU information of my system, such as the processor architecture, number of CPU cores, CPU model, and other hardware details. This helped me understand the processor resources available.

Ps:

```
pavani@DESKTOP-380LEIN:~$ ps
  PID TTY          TIME CMD
  363 pts/0        00:00:00 bash
  636 pts/0        00:00:00 ps
```


REPORT: The ps command displayed the processes currently running in my terminal. I observed the process IDs and command names of the active processes.

Top:

```
0.00 ps/0 00:00:00 ps
pavani@DESKTOP-380LEIN:~$ top
top - 09:05:15 up 7 min, 1 user, load average: 0.05, 0.05, 0.01
Tasks: 28 total, 1 running, 27 sleeping, 0 stopped, 0 zombie
%Cpu(s): 0.0 us, 0.0 sy, 0.0 ni,100.0 id, 0.0 wa, 0.0 hi, 0.0 si, 0.0 st
MiB Mem : 3803.6 total, 2402.1 free, 555.3 used, 982.8 buff/cache
MiB Swap: 1024.0 total, 1024.0 free, 0.0 used. 3248.3 avail Mem
```

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
1	root	20	0	24384	15472	11616	S	0.0	0.4	0:00.76	systemd
2	root	20	0	3180	2208	2072	S	0.0	0.1	0:00.01	init-systemd(Ub
6	root	20	0	3180	2040	1940	S	0.0	0.1	0:00.00	init
46	root	19	-1	50396	16892	15592	S	0.0	0.4	0:00.21	systemd-journal
77	systemd+	20	0	22540	14656	12032	S	0.0	0.4	0:00.07	systemd-resolve
82	root	20	0	35560	12588	9316	S	0.0	0.3	0:00.19	systemd-udevd
144	root	20	0	159348	1836	1508	S	0.0	0.0	0:00.00	snappyfuse
150	root	20	0	454144	11612	1556	S	0.0	0.3	0:02.29	snappyfuse
177	root	20	0	2888	2016	1884	S	0.0	0.1	0:00.05	chronyd-starter
178	root	20	0	4472	3168	2924	S	0.0	0.1	0:00.01	cron
179	message+	20	0	8816	5508	4824	S	0.0	0.1	0:00.05	dbus-daemon
183	root	20	0	44092	29820	14676	S	0.0	0.8	0:00.49	networkd-dispat
186	root	20	0	2220748	38668	25020	S	0.0	1.0	0:00.85	snappyd
187	root	20	0	18444	9544	8292	S	0.0	0.2	0:00.07	systemd-logind
190	root	20	0	1857652	15260	12616	S	0.0	0.4	0:00.15	wsl-pro-service
207	syslog	20	0	220548	6020	4668	S	0.0	0.2	0:00.06	rsyslogd
210	root	20	0	5492	2828	2644	S	0.0	0.1	0:00.01	agetty
222	root	20	0	123456	32588	17976	S	0.0	0.8	0:00.41	unattended-upgr
228	_chrony	20	0	24252	11904	7304	S	0.0	0.3	0:00.10	chronyd
249	_chrony	20	0	12092	2512	1868	S	0.0	0.1	0:00.01	chronyd
361	root	20	0	3188	1112	980	S	0.0	0.0	0:00.00	SessionLeader
362	root	20	0	3204	1248	1108	S	0.0	0.0	0:00.01	Relay(363)
363	pavani	20	0	6268	5600	3928	S	0.0	0.1	0:00.06	bash
364	root	20	0	8648	5576	4732	S	0.0	0.1	0:00.01	login
432	pavani	20	0	22340	13656	11032	S	0.0	0.4	0:00.10	systemd
434	pavani	20	0	22912	4080	2076	S	0.0	0.1	0:00.00	(sd-pam)
463	pavani	20	0	6136	5548	3992	S	0.0	0.1	0:00.02	bash
646	pavani	20	0	8196	6084	3960	R	0.0	0.2	0:00.02	top

REPORT: The top command displayed the live status of the system. I observed the CPU usage, memory usage, running processes, and system performance updating continuously.